

Procedimiento de analisis estatico profundo de ficheros EXE maliciosos

Guia de trabajo para triage, analisis PE, deteccion de packing/ofuscacion, extraccion de artefactos, desensamblado y preparacion de emulacion o analisis dinamico.

Este documento esta pensado para analizar muestras Windows Portable Executable, principalmente .exe, aunque gran parte del procedimiento tambien aplica a .dll, .scr, .sys y otros binarios PE. El objetivo es obtener una conclusion estatica solida sin ejecutar la muestra fuera de un entorno controlado.

Indice

- 0. Flujo general
- 1. Informacion general de la muestra
- 2. Triage estatico inicial
- 3. Validacion de estructura PE
- 4. Deteccion de packing, ofuscacion y anomalias
- 5. Busqueda de cadenas e IOCs
- 6. Imports, APIs y capacidades
- 7. Recursos, overlay y payloads embebidos
- 8. Analisis con desensamblador/decompilador
- 9. Extraccion de configuracion y desofuscacion
- 10. Clasificacion de hallazgos
- 11. Conclusion estatica prudente
- 12. Cuando pasar a emulacion o analisis dinamico

0. Flujo general

```
identificacion -> estructura PE -> packing/ofuscacion -> strings/imports -> recursos/overlay  
-> desensamblado -> configuracion -> conclusion estatica -> emulacion/dinamico si procede
```

El analisis de un EXE malicioso debe avanzar de lo general a lo concreto. Primero se verifica que el fichero es realmente un PE y se documentan sus hashes. Despues se revisan cabeceras, secciones, imports, cadenas, recursos y posibles payloads embebidos. Solo cuando exista evidencia suficiente se pasa a desensamblado, extraccion de configuracion o emulacion.

1. Informacion general de la muestra

1.1 Identificacion del fichero

Se confirma el tipo de fichero y se comprueba si la extension coincide con el contenido real.

```
file muestra.exe  
exiftool muestra.exe  
trid muestra.exe  
xxd -l 64 muestra.exe
```

Cabeceras habituales:

4D 5A	-> MZ / DOS header
50 45 00 00	-> PE signature
D0 CF 11 E0 A1 B1 1A E1	-> OLE/CFBF, no PE directo
50 4B 03 04	-> ZIP/OOXML/JAR/APK, no PE directo

1.2 Hashes y footprinting

Se calculan hashes criptograficos y hashes de similitud para identificar la muestra y compararla con variantes.

```
md5sum muestra.exe
sha1sum muestra.exe
sha256sum muestra.exe
ssdeep muestra.exe
tlsh -c muestra.exe      # si TLSH esta instalado
```

En Windows/Sysinternals tambien es util obtener informacion de firma, version y hashes:

```
sigcheck.exe -a -h -i muestra.exe
```

1.3 Consultas externas

De forma opcional se consultan servicios externos como VirusTotal, MalwareBazaar, Hybrid Analysis, ANY.RUN, Joe Sandbox o CAPE Sandbox. Estas fuentes deben usarse como contexto y no como sustituto del analisis propio.

Conviene anotar detecciones, nombres de familia, relaciones con otros hashes, firmas, URLs, dominios y comportamiento reportado. Despues esos datos se intentaran confirmar localmente.

2. Triage estatico inicial

2.1 Identificacion PE y arquitectura

Se identifica si el binario es PE32 o PE32+, su arquitectura, subsistema y punto de entrada.

```
pefile muestra.exe
python -m pefile muestra.exe
objdump -x muestra.exe | head -80
rabin2 -I muestra.exe
pedump muestra.exe      # si esta disponible
```

Campos importantes:

- Machine: x86, x64, ARM, etc.
- Subsystem: GUI, console, driver, native.
- AddressOfEntryPoint: direccion relativa de inicio.
- ImageBase: base preferida de carga.
- NumberOfSections: numero de secciones.
- TimeDateStamp: fecha de compilacion declarada, que puede estar manipulada.

2.2 Herramientas de triage recomendadas

En esta fase se busca una primera fotografia de la muestra sin entrar todavia en desensamblado profundo.

```
diec muestra.exe      # Detect It Easy en consola
pecheck.py muestra.exe
pestudio.exe muestra.exe
capa muestra.exe
floss muestra.exe > floss.txt
strings -a muestra.exe > strings_ascii.txt
strings -a -el muestra.exe > strings_utf16.txt
```

capa es especialmente util para obtener una vista inicial de capacidades: networking, persistence, process injection, anti-debugging, crypto, service creation, etc. FLOSS ayuda a recuperar cadenas ofuscadas comunes.

2.3 Seguridad operacional

No debe ejecutarse el binario en el sistema anfitrión. Si se requiere analisis dinamico, debe realizarse en una VM aislada, con snapshot previo, red controlada y herramientas de monitorizacion.

3. Validacion de estructura PE

3.1 Cabeceras principales

Se revisan DOS header, PE header, Optional Header y Data Directories. El objetivo es detectar incoherencias estructurales, corrupcion intencionada o trucos para confundir parsers.

```
import pefile
pe = pefile.PE("muestra.exe")
print("EntryPoint:", hex(pe.OPTIONAL_HEADER.AddressOfEntryPoint))
print("ImageBase:", hex(pe.OPTIONAL_HEADER.ImageBase))
print("Subsystem:", pe.OPTIONAL_HEADER.Subsystem)
```

```
print("Sections:", pe.FILE_HEADER.NumberOfSections)
print("Timestamp:", hex(pe.FILE_HEADER.TimeDateStamp))
```

3.2 Secciones

Se analizan nombres, permisos, tamanos, entropia, raw size, virtual size y si contienen codigo, datos o recursos.

```
pecheck.py -l S muestra.exe
rabin2 -S muestra.exe
objdump -h muestra.exe
```

Indicadores sospechosos:

- Secciones con nombres no habituales: UPX0, UPX1, .packed, .aspack, .vmp, .themida o nombres aleatorios.
- Secciones con permisos RWX.
- Entropia alta en secciones de codigo o datos.
- VirtualSize mucho mayor que RawSize.
- Entry point dentro de una seccion no habitual.
- Secciones sin nombre o con tamanos inconsistentes.

3.3 Data directories

Se revisan import table, export table, resources, relocation table, TLS callbacks, debug directory y exception directory.

```
pecheck.py -l D muestra.exe
objdump -p muestra.exe
rabin2 -i muestra.exe
```

Los TLS callbacks son importantes: un malware puede ejecutar codigo antes del entry point declarado. Tambien hay que revisar si la import table esta ausente, vacia o es anormalmente pequena.

3.4 Firma digital y Rich header

La firma digital puede aportar informacion de confianza o abuso de certificados. El Rich header puede ayudar a inferir entorno de compilacion, aunque tambien puede estar borrado o falsificado.

```
sigcheck.exe -i -h muestra.exe
pecheck.py -l R muestra.exe
```

Una firma invalida, caducada, revocada o perteneciente a un editor no esperado no prueba por si sola que el fichero sea malicioso, pero es un indicador de interes.

4. Deteccion de packing, ofuscacion y anomalias

4.1 Deteccion de packers

Se buscan packers, protectores y empaquetadores comunes.

```
diac muestra.exe
peid muestra.exe          # si esta disponible
pecheck.py muestra.exe
capa muestra.exe
```

Indicadores habituales de packing:

- Muy pocas imports, por ejemplo solo LoadLibrary y GetProcAddress.
- Entropia alta en secciones principales.
- Entry point en seccion con nombre sospechoso.
- Secciones UPX0/UPX1, .packed o nombres aleatorios.
- Gran diferencia entre tamano en disco y tamano en memoria.
- Overlay grande o recursos con alta entropia.

4.2 Entropia

La entropia ayuda a identificar datos comprimidos o cifrados. No es prueba definitiva, pero orienta el analisis.

```
pecheck.py -l E muestra.exe
binwalk -E muestra.exe
```

Una entropia alta puede indicar packing, cifrado o compresion. Una entropia baja puede aparecer en padding, zonas vacias o datos estructurados.

4.3 UPX y desempaquetado controlado

Si se detecta UPX, puede intentarse desempaquetado sobre una copia de la muestra.

```
cp muestra.exe muestra_upx_test.exe
upx -t muestra_upx_test.exe
upx -d muestra_upx_test.exe -o muestra_unpacked.exe
```

Si el desempaquetado falla, no hay que forzarlo sin documentar el error: algunos malware modifican UPX o usan packers personalizados.

4.4 Ofuscacion de cadenas

Si strings no muestra informacion util, se prueban herramientas orientadas a cadenas ofuscadas.

```
strings -a muestra.exe | head
strings -a -el muestra.exe | head
floss muestra.exe > floss.txt
xorsearch muestra.exe "http"
xorsearch muestra.exe "kernel32"
xorsearch muestra.exe "This program"
```

5. Busqueda de cadenas e IOCs

5.1 Busqueda generica de cadenas

Antes de buscar IOCs concretos, se realiza una busqueda generica en ASCII y UTF-16LE.

```
strings -a -n 4 muestra.exe | grep -Ei "http|https|www\.|\.exe|\.dll|cmd|powershell|wscript|
mshta|rundll32|regsvr32|download|url|kernel|loadlibrary|getProcAddress|virtualalloc|virtualp
rotect|createremotethread|writeprocessmemory|run|currentversion|appdata|temp|programdata|pub
lic"

strings -a -el -n 4 muestra.exe | grep -Ei "http|https|www\.|\.exe|\.dll|cmd|powershell|wscr
ipt|mshta|rundll32|regsvr32|download|url|kernel|loadlibrary|getProcAddress|virtualalloc|virt
ualprotect|createremotethread|writeprocessmemory|run|currentversion|appdata|temp|programdata
|public"
```

5.2 Cadenas de interes

- Red: http, https, ws2_32, wininet, winhttp, InternetOpen, InternetConnect, HttpSendRequest.
- Ejecucion: CreateProcess, WinExec, ShellExecute, cmd.exe, powershell.exe, wscript.exe.
- Inyeccion: VirtualAllocEx, WriteProcessMemory, CreateRemoteThread, NtMapViewOfSection.
- Persistencia: Run, RunOnce, Services, Startup, Scheduled Tasks, AppInit_DLLs.
- Evasion: IsDebuggerPresent, CheckRemoteDebuggerPresent, NtQueryInformationProcess, Sleep.
- Credenciales: CryptUnprotectData, CredEnumerate, referencias a navegadores, wallets o clientes de correo.

5.3 Extraccion de IOCs

Se deben separar cadenas tecnicas de IOCs reales. Un nombre de API no es un IOC. Una URL, dominio, IP, ruta de dropped file o mutex si pueden serlo.

```
grep -Eai "https?://[^\ ]+" strings_ascii.txt
grep -Eai "([a-zA-Z0-9-]+\.)+[a-zA-Z]{2,}" strings_ascii.txt
grep -Eai "[A-Za-z]:\\|\\%APPDATA%|\\%TEMP%|\\%PUBLIC%|ProgramData" strings_ascii.txt
```

6. Imports, APIs y capacidades

6.1 Tabla de imports

La tabla de imports muestra APIs enlazadas estaticamente. Es un punto fundamental para inferir capacidades.

```
objdump -p muestra.exe | less
rabin2 -i muestra.exe
pecheck.py -l I muestra.exe
```

Si hay pocas imports, puede haber packing o resolucion dinamica de APIs mediante LoadLibrary/GetProcAddress o hashing de APIs.

6.2 Capacidades con capa

capa detecta patrones de comportamiento a partir del codigo y las imports.

```
capa muestra.exe > capa.txt
capa -vv muestra.exe > capa_verbose.txt
```

Revisar especialmente capacidades como persistencia, inyeccion, keylogging, enumeracion del sistema, comunicacion de red, crypto, anti-debugging y creacion de servicios.

6.3 APIs de interes por comportamiento

- Red: WinHttpOpen, WinHttpConnect, InternetOpen, InternetReadFile, URLDownloadToFile, socket, connect.
- Proceso: CreateProcess, WinExec, ShellExecute, CreateToolhelp32Snapshot.
- Inyeccion: OpenProcess, VirtualAllocEx, WriteProcessMemory, CreateRemoteThread, QueueUserAPC.
- Persistencia: RegSetValue, CreateService, schtasks, carpeta Startup.
- Ficheros: CreateFile, WriteFile, CopyFile, MoveFile, DeleteFile.
- Evasion: IsDebuggerPresent, Sleep, NtQueryInformationProcess, GetTickCount.

7. Recursos, overlay y payloads embebidos

7.1 Analisis de recursos

Los recursos pueden contener iconos, manifiestos, configuraciones, payloads cifrados o binarios embebidos.

```
rabin2 -R muestra.exe
pecheck.py -l r muestra.exe
```

En Windows se puede usar Resource Hacker:

```
ResourceHacker.exe -open muestra.exe -save recursos.rc -action extract
```

En Linux pueden usarse herramientas como wrestool si estan disponibles:

```
wrestool -l muestra.exe
wrestool -x muestra.exe > recursos_extraidos.bin
```

7.2 Busqueda de PE embebidos

Se buscan cabeceras MZ dentro del fichero y de los recursos extraidos.

```
grep -aob "MZ" muestra.exe
pecheck.py -l P muestra.exe
binwalk muestra.exe
```

Si se localiza una cabecera MZ desplazada, se puede hacer carving y analizar el resultado como un PE independiente.

```
dd if=muestra.exe of=payload_carved.exe bs=1 skip=OFFSET status=none
file payload_carved.exe
pecheck.py payload_carved.exe
```

7.3 Overlay

El overlay es contenido que aparece despues del final logico de las secciones PE. Puede contener configuracion, un payload adicional, datos cifrados o basura antianalisis.

```
pecheck.py -l 0 muestra.exe

import pefile
pe = pefile.PE("muestra.exe")
print(pe.get_overlay_data_start_offset())
```

Si existe overlay, debe extraerse y analizarse con file, strings, xxd, binwalk y entropia.

8. Analisis con desensamblador/decompilador

8.1 Preparacion en Ghidra, IDA o Cutter

Se carga el PE en un desensamblador y se deja completar el analisis automatico. Despues se revisan entry point, imports, strings, xrefs y funciones sospechosas.

```
Ghidra:
- Importar muestra.exe.
- Analizar con opciones por defecto.
- Revisar entry point.
- Revisar Symbol Tree -> Imports.
- Revisar Defined Strings.
- Seguir Xrefs hacia cadenas, APIs y recursos.
```

8.2 Zonas prioritarias

- Entry point y codigo cercano.
- WinMain/main o funcion inicial real tras runtime.
- Llamadas a LoadLibrary/GetProcAddress.
- Funciones con muchas referencias a strings.
- Funciones que acceden a registro, red, procesos o ficheros.
- Rutinas de decryption/decompression.
- TLS callbacks, si existen.
- Codigo que resuelve APIs por hash o recorre PEB/export table.

8.3 Resolucion dinamica de APIs

Si el binario no usa imports claras, buscar patrones de resolucion dinamica: acceso a PEB, recorrido de export table, hashing de nombres de API o llamadas indirectas.

```
Patrones habituales:
- mov eax, fs:[0x30]      # PEB en x86
- mov rax, gs:[0x60]      # PEB en x64
- acceso a e_lfanew [base+0x3c]
- recorrido de IMAGE_EXPORT_DIRECTORY
- comparacion/hash de nombres exportados
- llamadas indirectas call eax/call rax/call [reg]
```

8.4 Analisis de cadenas y configuracion

Cada cadena relevante debe seguirse con Xrefs. No basta con verla en strings: hay que comprobar donde se usa.

```
Preguntas clave:
- Que funcion referencia la cadena?
- Es dato real o basura?
- Se descifra en runtime?
- Se usa como URL, ruta, mutex, clave de registro, user-agent o comando?
- Hay estructuras de configuracion alrededor?
```

9. Extraccion de configuracion y desofuscacion

9.1 Localizacion de configuracion

Muchos malware almacenan configuracion en recursos, overlay, secciones cifradas, arrays de bytes o cadenas codificadas.

```
Buscar:
- URLs, dominios, IPs, puertos.
- User-Agent.
- Claves XOR/RC4/AES.
- Mutex.
- Rutas de instalacion.
- Nombres de servicios o tareas programadas.
- Offsets de configuracion en recursos u overlay.
```

9.2 Decodificacion manual o con script

Si se identifica una codificacion simple, se puede crear un script de apoyo. El script debe documentar input, algoritmo y output.

```
# Ejemplo XOR simple en Python
blob = open("blob.bin", "rb").read()
for key in range(256):
    out = bytes(b ^ key for b in blob)
    if b"http" in out or b"MZ" in out:
```

```
print("key", hex(key), out[:200])
```

9.3 Validacion de configuracion

Una configuracion extraida debe validarse con referencias cruzadas en el codigo. Si una URL aparece en un blob pero no hay codigo que la use, debe describirse como indicio, no como comportamiento confirmado.

10. Clasificacion de hallazgos

10.1 Hallazgos estructurales

```
PE valido
Arquitectura x86/x64
Secciones sospechosas
Overlay
Recursos embebidos
TLS callbacks
Firma digital
Rich header
Packing probable
```

10.2 Hallazgos tecnicos

```
Resolucion dinamica de APIs
Inyeccion de procesos
Persistencia
Comunicacion de red
Descarga de payload
Cifrado/compresion
Anti-debugging / anti-VM
Robo de informacion
Ejecucion de comandos
```

10.3 IOCs

```
Hashes
Imphash
TLSH / ssdeep
Dominios
URLs
IPs
Rutas de fichero
Mutexes
Claves de registro
Nombres de servicios
Nombres de tareas programadas
User-Agent
```

11. Conclusion estatica prudente

La conclusion debe distinguir entre evidencia estatica, hipotesis y comportamiento confirmado. No se debe afirmar ejecucion real de una API si solo se ha visto su nombre en cadenas.

```
Ejemplo:
El analisis estatico identifica un ejecutable PE con secciones de alta entropia, imports
reducidas y cadenas relacionadas con resolucion dinamica de APIs. Se observan referencias a
funciones de red, rutas de escritura y posibles mecanismos de persistencia. Estos elementos
son compatibles con un downloader o loader. No obstante, la creacion real de procesos,
conexiones de red o escritura de payloads debera confirmarse mediante emulacion o analisis
dinamico controlado.
```

12. Cuando pasar a emulacion o analisis dinamico

Se pasa a emulacion o dinamico cuando el analisis estatico no basta para confirmar comportamiento o cuando hay packing, cifrado de configuracion o resolucion dinamica compleja.

12.1 Herramientas utiles

```
Dinamico / monitorizacion:
- Procmon
- Process Explorer
- Regshot
- Autoruns
- TCPView
- Wireshark
- FakeNet-NG
```

- INetSim

Debugging / emulacion:

- x32dbg / x64dbg
- API Monitor
- Speakeasy
- Qiling
- Unicorn
- CAPE Sandbox

12.2 Preparacion minima

1. VM aislada y snapshot previo.
2. Red controlada o simulada.
3. Herramientas de monitorizacion abiertas antes de ejecutar.
4. Hashes y copia de la muestra preservados.
5. Plan de indicadores a confirmar: procesos, ficheros, registro, red, mutexes, inyeccion.

12.3 Relacion con el analisis estatico

El analisis dinamico debe responder preguntas derivadas del estatico:

- Se resuelven realmente las APIs sospechosas?
- Se crea el fichero observado en strings?
- Se conecta a la URL o dominio identificado?
- Se establece persistencia?
- Se inyecta codigo en otro proceso?
- Se descifra una configuracion o payload adicional?